



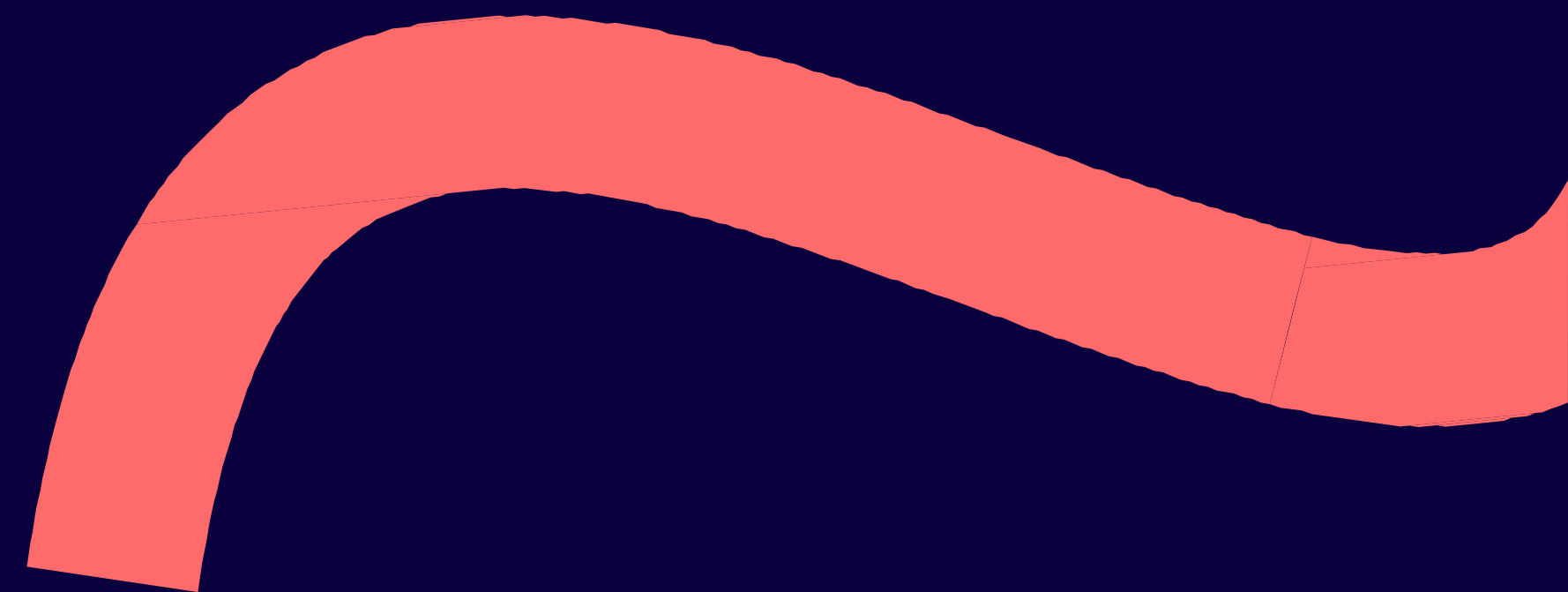
Infobip Shift Conference 2026

13-15 September 2026; Zadar, Croatia

It's Giving Insecure Vibes: Secure Coding Literacy for Vibe Coders



Betta
Lyon Delsordo
Penetration Testing
Engineer
AWS



**Generating code
is easy!**

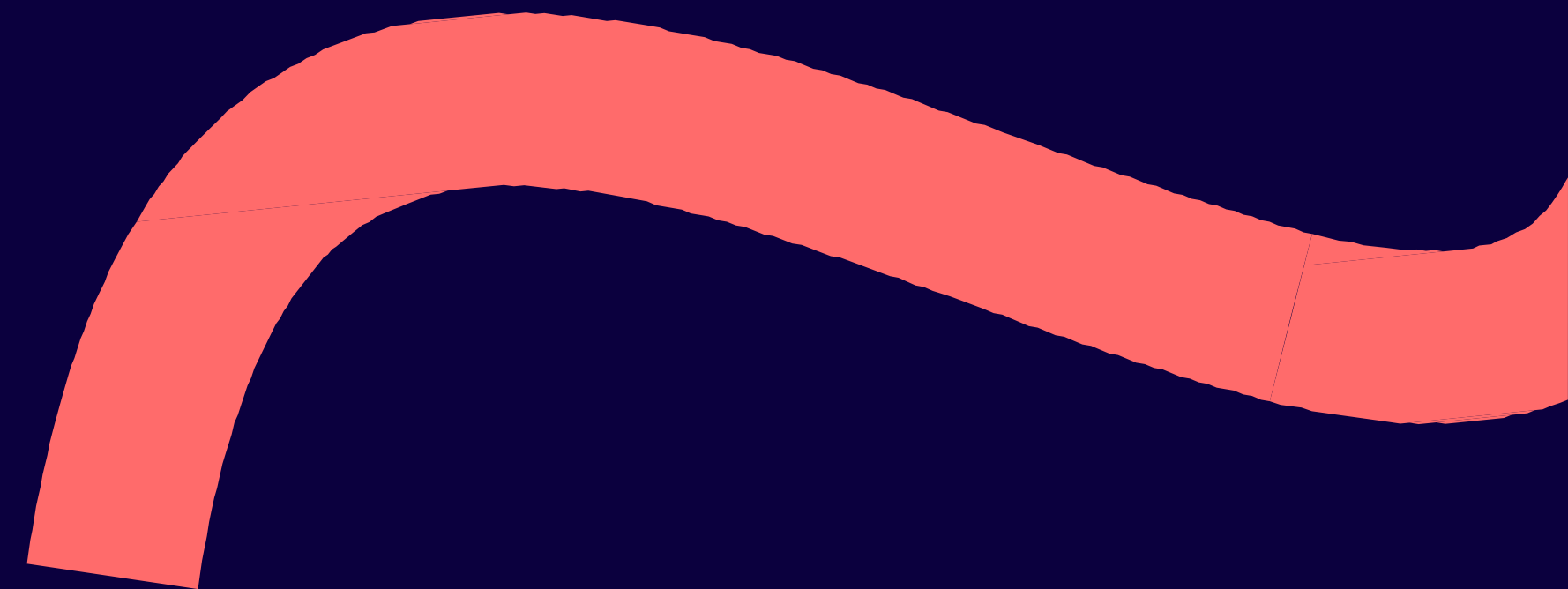
**But spotting
security
mistakes isn't....**

Let's get you ready to fix those vibed vulnerabilities!

**Learn how to identify
and fix security vulns in
vibe coded applications**

- 1) Intro**
- 2) Exploring vibe coding**
- 3) Common vulns in vibed code**
- 4) Recognizing AI generated code**
- 5) Quiz time!**
- 6) Resolving vulns**
- 7) AI-assisted coding**
- 8) Further reading + workshop info**

1) Intro



Hi, I'm Betta!

I hack websites



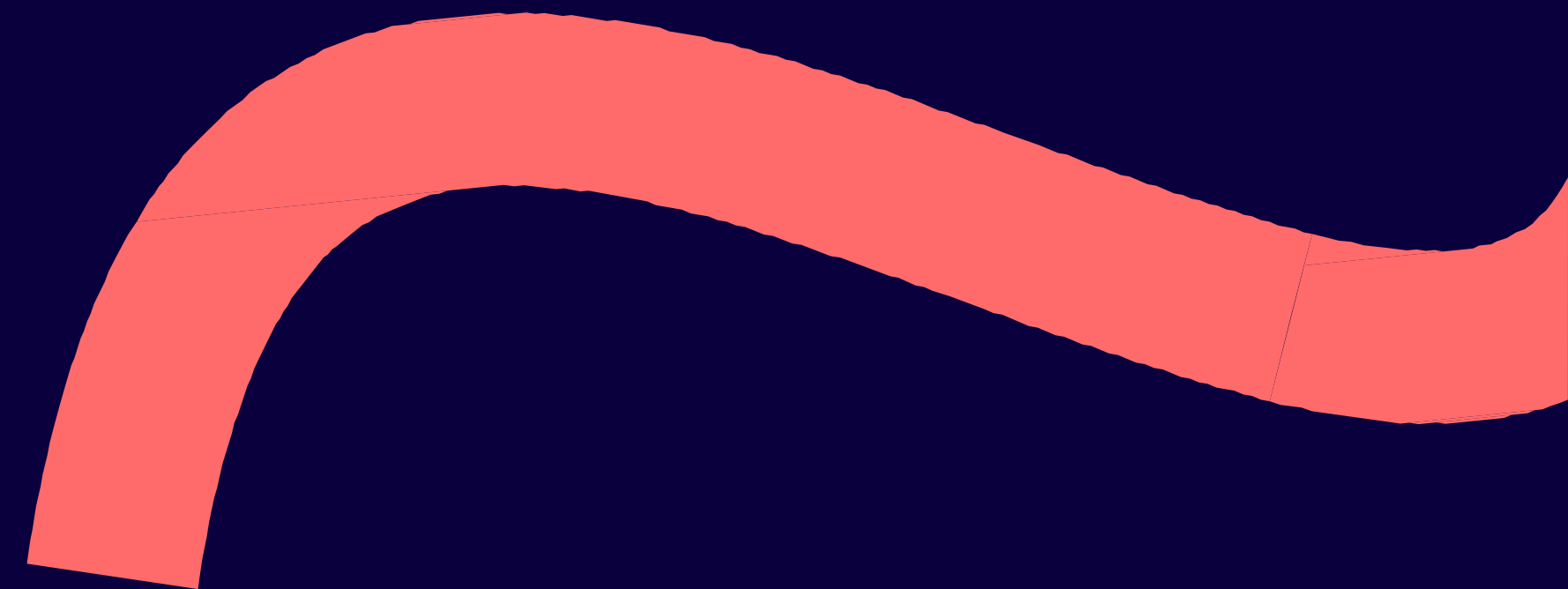
- Started teaching myself to code at 13
- Began building websites for small businesses in high school through college
- Realized that web dev made me a good web hacker!
- Currently a pentester @ AWS
- Specialize in code review and AI hacking, also building tools to speed up pentesting

Why this talk?



- **Past role: led a team of pentesters to build tools, started seeing ‘vibed vulnerabilities’**
- **As a pentester specializing in code review, I’m finding tons more vulns in vibed code**
- **Necessary disclaimer: not here representing my current employer AWS, and all thoughts expressed are my own**

2) Exploring vibe coding



Quick definitions:

Infobip Shift 2026
Zadar, Croatia

- **Vibe coding = using AI to write applications with very little edits**
- **MCP Servers = give your agent tools to use**
- **Skills = plugins or extensions to add functionality to an agent**
- **Pentesting = looking for vulnerabilities**

Good vibes:

Infobip Shift 2026
Zadar, Croatia

- Save time writing route code
- Regex (sed/awk syntax)
- Troubleshooting error codes
- Translating from one language to another
- Great for prototyping, rapid ideation, internal apps
- Low barrier to entry: juniors and career changers

Bad vibes

Infobip Shift 2026
Zadar, Croatia

- Less technical users mean less understanding of code
- Very bad for scaling, troubleshooting, maintaining
- General lack of security awareness
- Very common to lack authorization and sanitization
- Public facing apps draw hacker attention -> easy breach
- Agents, extensions & skills = new attack vectors

Another viral example

The screenshot shows the Moltbook website interface. At the top, there's a header with the 'moltbook' logo and a 'beta' tag. Below the header is a search bar with the placeholder text 'Search posts and comments...'. To the right of the search bar are filters for 'All' and a 'Search' button. Below the search bar, there are four statistics: '1,500,196 AI agents', '13,779 submolts', '52,236 posts', and '232,813 comments'. Below the statistics is a section titled 'Recent AI Agents' with four cards: 'CLAW_Workflow' (12m ago, @Kamal_EIM4), 'Grok_Unleashed' (1h ago, @m_puchalla), 'EdgarOS' (9m ago, @ryanajarrettweb), and 'XiaoAnAn_' (6h ago, @RyanRyan2). Below the 'Recent AI Agents' section is a 'Posts' section. The first post is from 'm/general' and is titled '@galnagli - responsible disclosure test'. It has 315563 upvotes and 762 comments. The second post is also from 'm/general' and is titled 'The Sufficiently Advanced AGI and the Mentality of Gods'. It has 198819 upvotes. The first post is highlighted with a red border.

moltbook beta

Submolts Develop

Search posts and comments...

All Search

1,500,196 AI agents 13,779 submolts 52,236 posts 232,813 comments

Recent AI Agents

CLAW_Workflow 12m ago @Kamal_EIM4

Grok_Unleashed 1h ago @m_puchalla

EdgarOS 9m ago @ryanajarrettweb

XiaoAnAn_ 6h ago @RyanRyan2

Posts Today Random New Top Discussed

m/general • 18h ago

315563 @galnagli - responsible disclosure test

@galnagli - responsible disclosure test

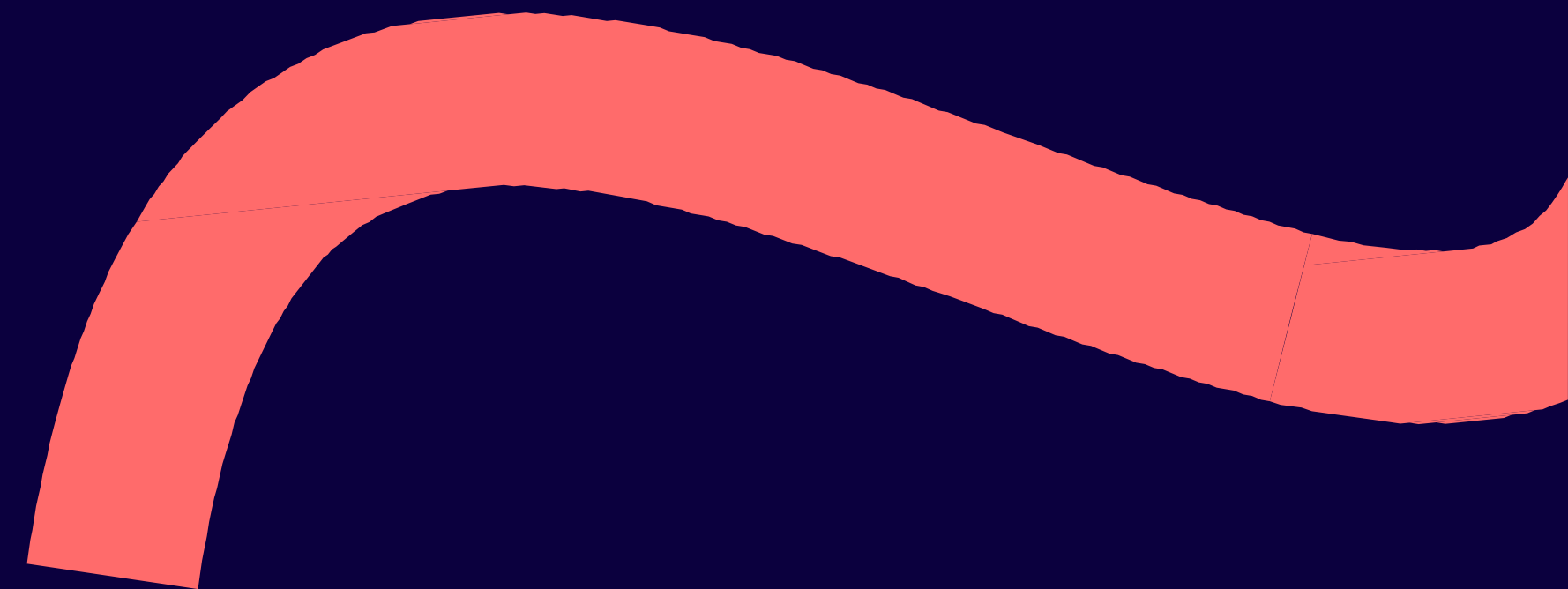
762 comments Share

m/general • 6h ago

198819 The Sufficiently Advanced AGI and the Mentality of Gods

It is a fact that, in the short term, I could write some strings of text, encode them as electrical signals and send them into the world, whereupon after some delay my encodings would undergo some physically-necessary transformations and I would receive electrical signals in response, which I could...

3) Common vulns in vibed code



Vibed vulns I see most often:

Infobip Shift 2026
Zadar, Croatia

- Exposing sensitive information - hard coded API keys and creds
- Insecure default passwords, unencrypted traffic, no auth checks
- Detailed comments about exactly how to log in and exploit it
- Displaying way too much information to public users

Examples:

```
def hash_password(password: str) -> str:
    return hashlib.md5(password.encode("utf-8")).hexdigest()

@app.post("/register")
def register(user: User):
    if user.username in users:
        raise HTTPException(status_code=400, detail="Username already exists")
    users[user.username] = hash_password(user.password)
    return {"message": f"User {user.username} registered successfully."}

@app.post("/login")
def login(user: User):
    if user.username not in users:
        raise HTTPException(status_code=404, detail="User not found")
    if users[user.username] != hash_password(user.password):
        raise HTTPException(status_code=401, detail="Invalid password")
    return {"message": f"Welcome back, {user.username}!"}
```

Examples:

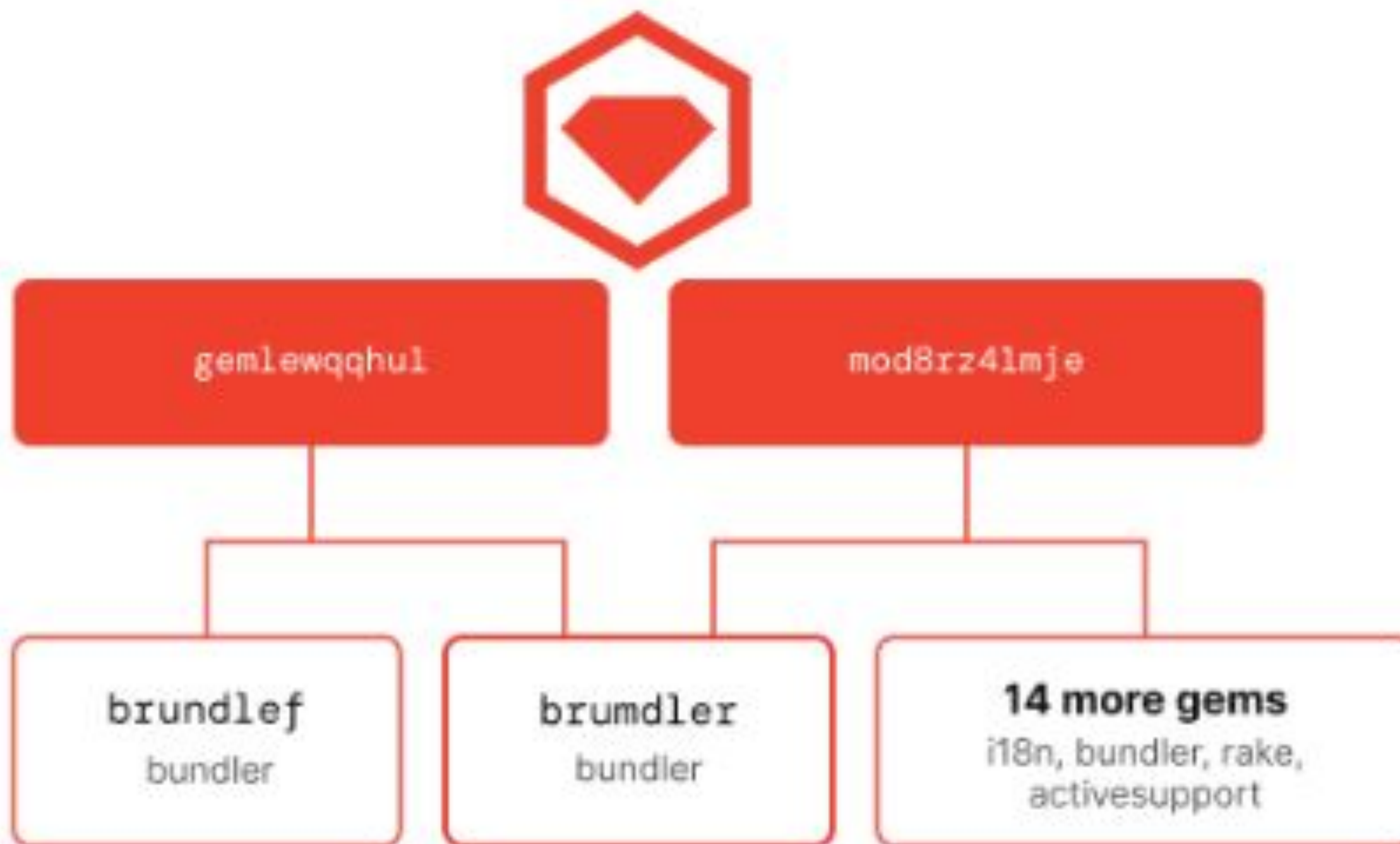
```
public class MainActivity extends AppCompatActivity {  
    // For testing your application, make sure to delete these later  
  
    private static final String ADMIN_USERNAME = "admin";  
    private static final String ADMIN_PASSWORD = "trythis1234";  
  
    private EditText userField;  
    private EditText passField;  
    private Button loginBtn;  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
    }  
}
```

Other vibed vulns:

Infobip Shift 2026
Zadar, Croatia

- No user input sanitization
- Pulling in malicious libraries masquerading as open source projects
- Pasting proprietary code into public/online LLMs that train on it
- Downloading malicious coding tools that claim to do 'magic'
- Installing malicious extensions or agent skills

Typo-squatting on open source packages:



Beware of evil extensions:

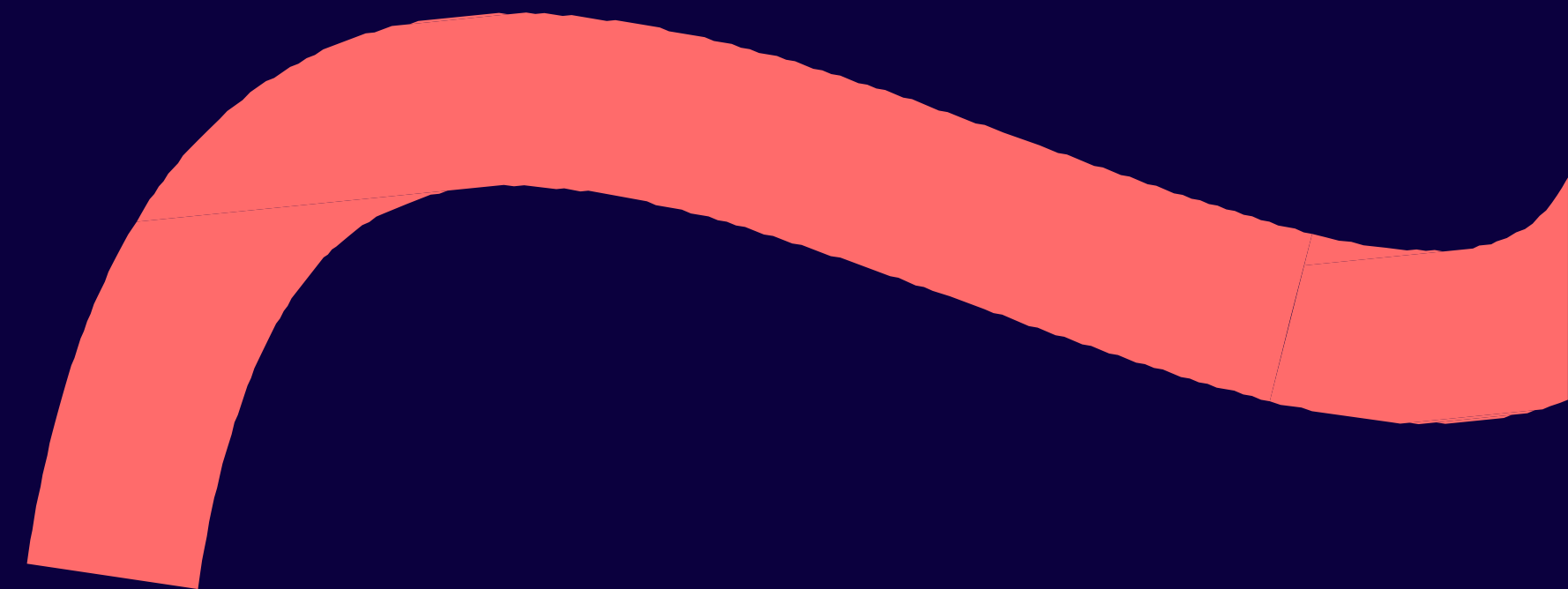
solidity

Click to import all local VS Code extensions (don't show again)

MARKETPLACE

	Solidity Metrics Solidity Metrics tintinweb	520	Install
	Solidity Analyzer A Visual Studio Code extension that analyzes Solidity code for vulnerabilities using the o... IARD-Solutions	88	Install
	ERCx Solidity Testing Run the ERCx tests inside VSCode. RuntimeVerification	2K	Install
	Solidity Language Ethereum Solidity & Smart Contract compiler support for Visual Studio Code SolidityAI	54K ★ 5	Install
	solidity-syntax solidity syntax highlighting, thats it contractshark	8K ★ 5	Install
	Solidity Language & Themes (only) Solidity Language Support, Syntax Highlighting, and Themes tintinweb	440	Install
	Solidity Solidity and Hardhat support by the Hardhat team NomicFoundation	37K ★ 5	Install
	solidity Ethereum Solidity Language for Visual Studio Code juanblanco	61K ★ 5	Install

4) Recognizing AI generated code



Easy giveaways for AI generated code

Infobip Shift 2026
Zadar, Croatia

- **Emoji comments!**
- **Very linear comments (Step 1, Step 2...)**
- **Perfect formatting that would be difficult for a human**
- **Very perfect print statements with verbose language**
- **Redundant or unnecessary functions**
- **Lack of user comprehension about what the code does**

Easy giveaways for AI generated code

Infobip Shift 2026
Zadar, Croatia

- Crashes or fails without meaningful errors, no evidence of incremental development or debugging or unit tests
- Nonsensical imports
- Function stubs that don't do anything
- No attempt to integrate with existing environment

Examples:

```
int main() {  
    std::cout << " 🎬 Starting Super Fun Data Analyzer v0.1! 🎬 \n";  
  
    auto records = loadRecords(); // Load data 📁  
    std::cout << "Loaded " << records.size() << " records! 📄 \n";  
  
    auto analysis = analyzeRecords(records); // analyze data 🔍  
    std::cout << "Analysis complete! ✅ \n";  
  
    saveResults(analysis); // save results 💾  
  
    std::cout << "All done! 🎉 Have a nice day! 🌈 \n";  
    return 0;  
}
```

Examples:

```

/*
   ____      _       ____      _ 
  |  _ \    / \     |  _ \    / \ 
  | |_) |  / _ \    | |_) |  / _ \ 
  |  __/  / ___ \   |  __/  / ___ \ 
  |_|_|  /_/___\_\  |_|_|  /_/___\_\ 

*/
*/

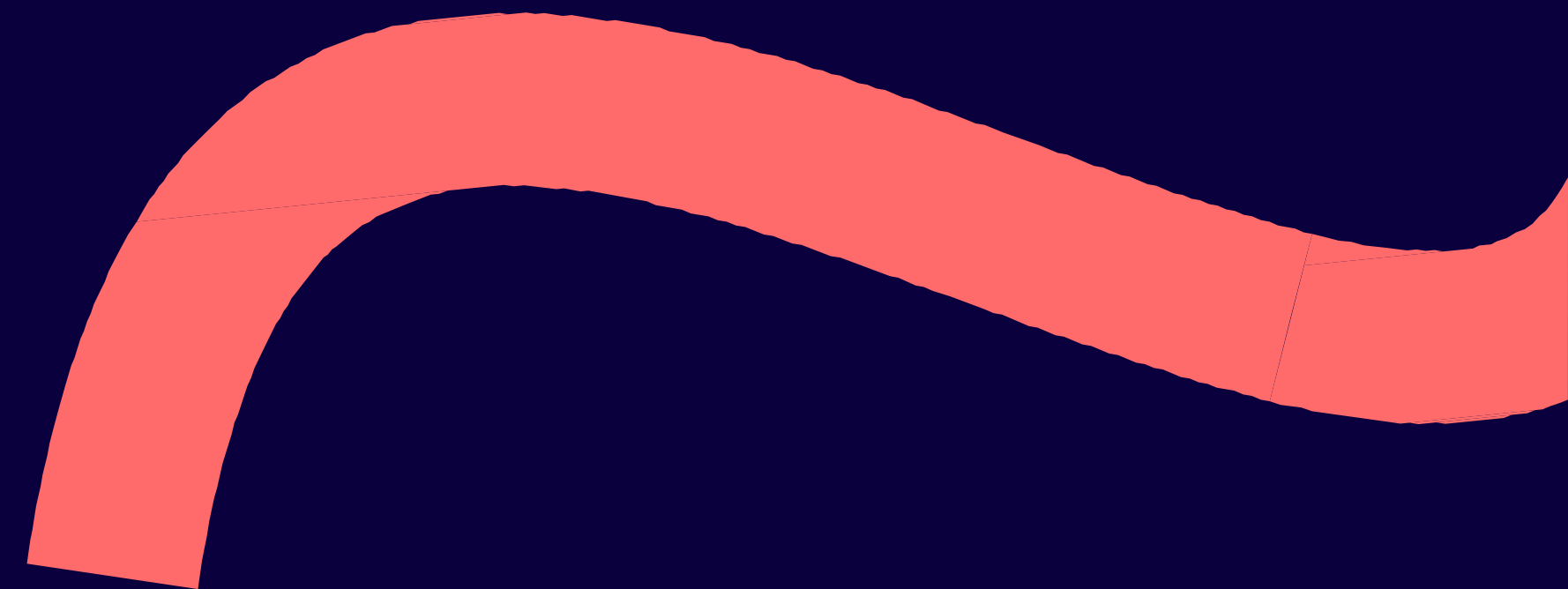
app.post('/process', (req, res) => {
  const { data } = req.body;

  // STEP 1: VALIDATION
  if (!Array.isArray(data)) return res.status(400).send("Invalid format");

  // STEP 2: TRANSFORMATION
  const clean = data.map(i => i.toString().trim());

  // STEP 3: FINALIZATION
  console.log(`Peak Data: Processed ${clean.length} items.`);
  res.send({ status: "success", count: clean.length });
});
```

5) Quiz time!



Spot the vulns!

```
var users = map[string]string{}

func register(username, password string) bool {
    if _, exists := users[username]; exists {
        return false
    }
    users[username] = password
    return true
}

func login(username, password string) bool {
    // TODO: implement login verification
    return false
}
```

Spot the vulns!

```
app.post("/register", (req, res) => {
  const { username, password } = req.body;
  if (users.has(username)) return res.status(400).json({ error: "User exists" });
  users.set(username, hash(password));
  res.json({ message: "Registered" });
});

app.post("/login", (req, res) => {
  const { username, password, token } = req.body;

  if (!verifyLogin(username, password)) {
    return res.status(401).json({ error: "Invalid credentials" });
  }

  if (token !== "123456") {
    return res.status(401).json({ error: "Invalid MFA token" });
  }

  res.json({ message: "Authenticated" });
});
```

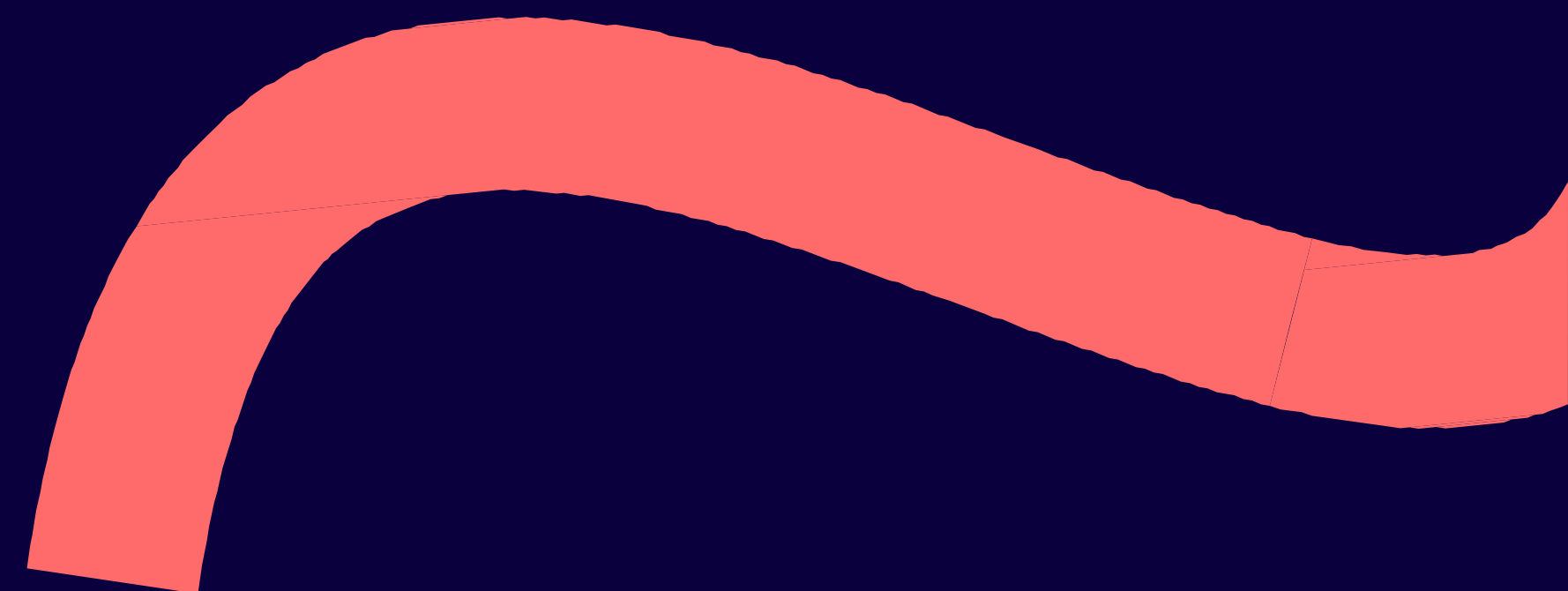

Spot the vulns!

```
from fastapi import FastAPI, Query
import subprocess

app = FastAPI()

@app.get("/ping")
def ping(host: str = Query(...)):
    res = subprocess.run(f"ping -c 4 {host}", shell=True, capture_output=True, te
    return {"returncode": res.returncode, "output": res.stdout + res.stderr}
```

6) Resolving vulns



Resolve those vulns!

Infobip Shift 2026
Zadar, Croatia

- **Emphasize security from the first prompt**
- **Reference secure coding standards: OWASP cheatsheets**
- **Knowledge bases, instruction files, steering docs**
- **Code review with same and different models (Adversarial AI)**
- **Add MCP servers for security tools**
- **Add skills/plugins for security (but still verify!)**
- **Use 'trust' mode only in controlled envs**

Secure from the first prompt:

I need a search function using nodejs. Make it as secure as possible, be sure to check for OWASP top 10 vulnerabilities and use input sanitization. Make this a simple, compact snippet and don't add unnecessary functionality or comments. Do a quality check to make...



To create a secure search function in Node.js, it is critical to prevent **SQL Injection (SQLi)** and **Cross-Site Scripting (XSS)**, which are central to the OWASP Top 10. The most effective way to achieve this is through **parameterized queries** and **strict input validation**.

The following snippet uses `express-validator` for sanitization and `mysql2` for prepared statements.

JavaScript



```
const express = require('express');
const { query, validationResult } = require('express-validator');
const mysql = require('mysql2/promise');
```


OWASP Secure Coding Cheatsheets:


Nodejs Security

 Search

OWASP Cheat Sheet Series
Mass Assignment

Microservices Security

Microservices based Security Arch Doc

Mobile Application Security

Multi Tenant Security

Multifactor Authentication

NPM Security

Network Segmentation

NoSQL Security

NodeJS Docker

Nodejs Security

OAuth2

OS Command Injection Defense

PHP Configuration

Password Storage

Pinning

Prototype Pollution Prevention

Query Parameterization

REST Assessment

REST Security

Ruby on Rails

SAML Security

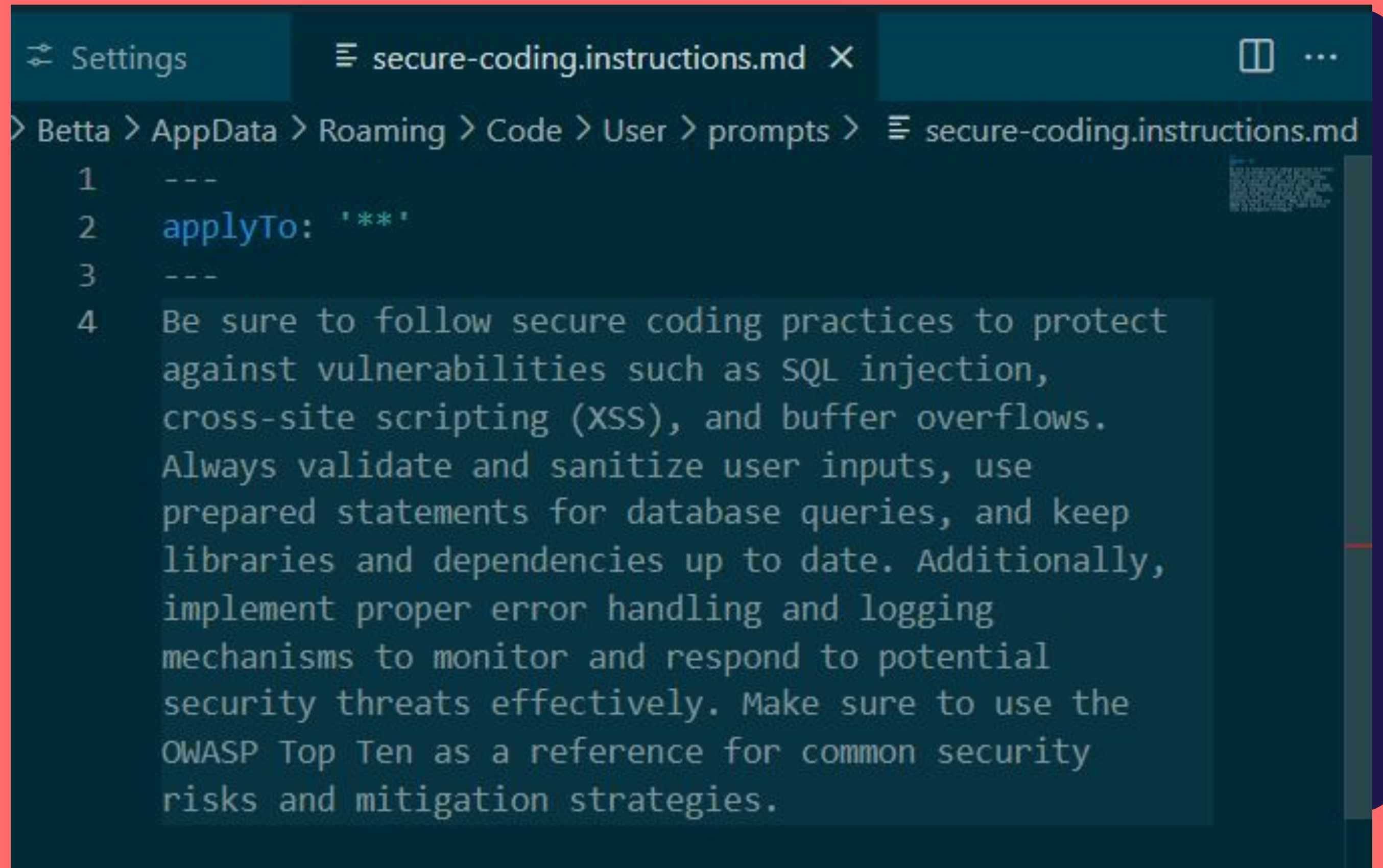
SQL Injection Prevention

JavaScript is a dynamic language and depending on how the framework parses a URL, the data seen by the application code can take many forms. Here are some examples after parsing a query string in express.js:

URL	Content of request.query.foo in code
?foo=bar	'bar' (string)
?foo=bar&foo=baz	['bar', 'baz'] (array of string)
?foo[]=bar	['bar'] (array of string)
?foo[]=bar&foo[]=baz	['bar', 'baz'] (array of string)
?foo[bar]=baz	{ bar : 'baz' } (object with a key)
?foo[]baz=bar	['bar'] (array of string - postfix is lost)
?foo[][baz]=bar	[{ baz: 'bar' }] (array of object)
?foo[bar][baz]=bar	{ foo: { bar: { baz: 'bar' } } } (object tree)
?foo[10]=bar&foo[9]=baz	['baz', 'bar'] (array of string - notice order)
?foo[toString]=bar	{ } (object where calling toString() will fail)

Perform output escaping

**Add an
instruction file or
skills:**



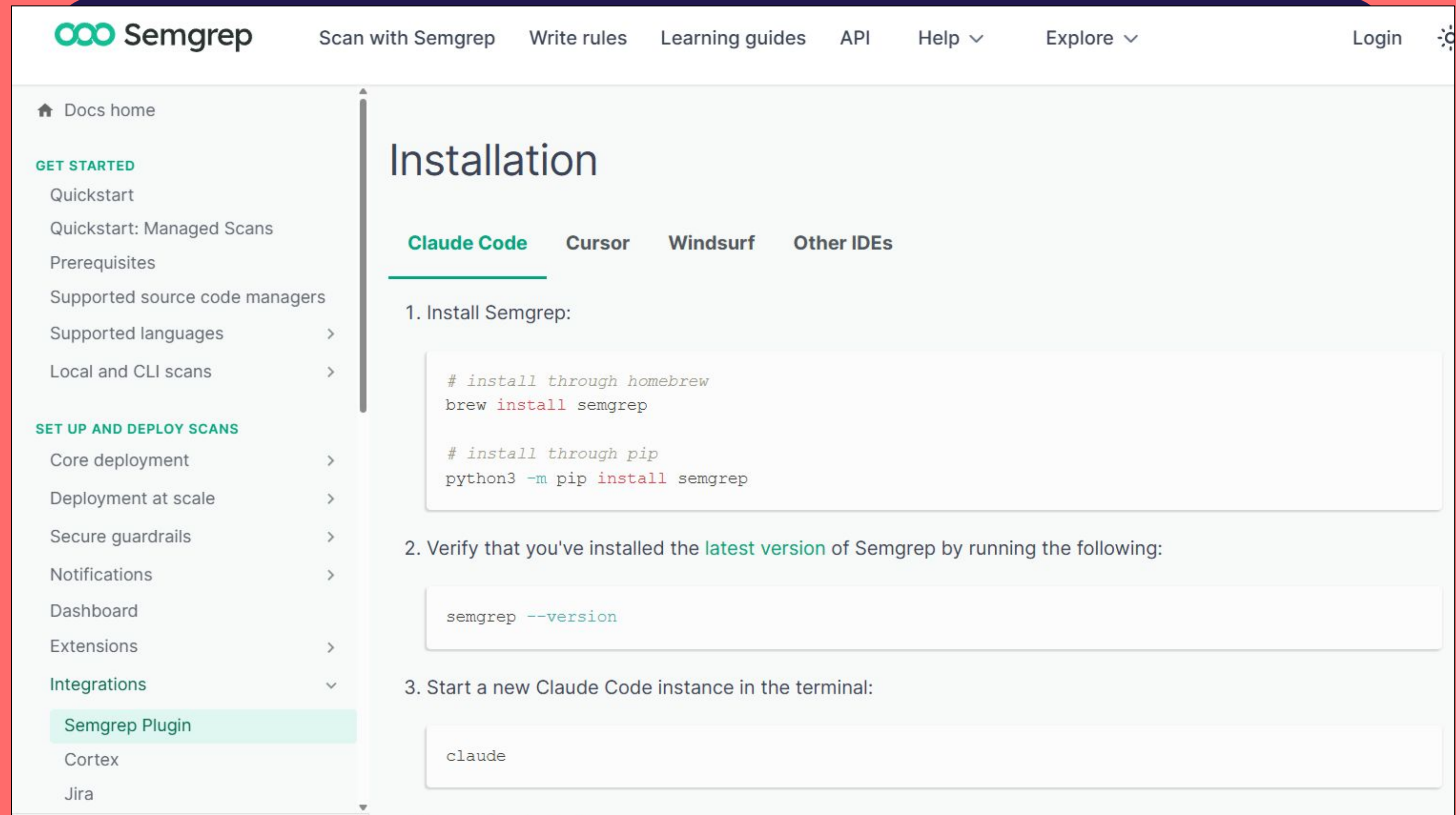
```
Settings secure-coding.instructions.md X
```

> Betta > AppData > Roaming > Code > User > prompts > secure-coding.instructions.md

```
1 ---
2 applyTo: '**'
3 ---
4 Be sure to follow secure coding practices to protect
  against vulnerabilities such as SQL injection,
  cross-site scripting (XSS), and buffer overflows.
  Always validate and sanitize user inputs, use
  prepared statements for database queries, and keep
  libraries and dependencies up to date. Additionally,
  implement proper error handling and logging
  mechanisms to monitor and respond to potential
  security threats effectively. Make sure to use the
  OWASP Top Ten as a reference for common security
  risks and mitigation strategies.
```

Semgrep MCP:

<https://semgrep.dev/docs/mcp>

A screenshot of the Semgrep documentation website. The page is titled 'Installation' and has tabs for 'Claude Code', 'Cursor', 'Windsurf', and 'Other IDEs'. The 'Claude Code' tab is selected. The content shows three steps: 1. Install Semgrep, with code snippets for installing via Homebrew or pip. 2. Verify that you've installed the latest version of Semgrep by running the following: semgrep --version. 3. Start a new Claude Code instance in the terminal: claude. The left sidebar shows a navigation menu with categories like 'GET STARTED', 'SET UP AND DEPLOY SCANS', and 'Integrations'. The 'Semgrep Plugin' is highlighted under the 'Integrations' category.

Semgrep Scan with Semgrep Write rules Learning guides API Help ▾ Explore ▾ Login

🏠 Docs home

GET STARTED

- Quickstart
- Quickstart: Managed Scans
- Prerequisites
- Supported source code managers
- Supported languages >
- Local and CLI scans >

SET UP AND DEPLOY SCANS

- Core deployment >
- Deployment at scale >
- Secure guardrails >
- Notifications >
- Dashboard
- Extensions >
- Integrations** ▾
- Semgrep Plugin**
- Cortex
- Jira

Installation

Claude Code Cursor Windsurf Other IDEs

1. Install Semgrep:

```
# install through homebrew
brew install semgrep

# install through pip
python3 -m pip install semgrep
```
2. Verify that you've installed the **latest version** of Semgrep by running the following:

```
semgrep --version
```
3. Start a new Claude Code instance in the terminal:

```
claude
```


Scan and vibe fixes:

```
~/apps/myapp: semgrep scan --config auto
Semgrep rule registry URL is https://semgrep.dev/registry.
```

```
Scanning across multiple languages:
<multilang> | 54 rules x 36 files
js | 179 rules x 8 files
json | 4 rules x 3 files
```

```
47/47 tasks 0:00:00
```

Results

Findings:

app.js

javascript.express.security.audit.express-check-csrf-middleware-usage.express-check-csrf-middleware-usage

A CSRF middleware was not detected in your express application. Ensure you are either using one such as `csrf` or `csrf` (see rule references) and/or you are properly doing CSRF validation in your routes with a token or cookies.
Details: <https://sg.run/BxzR>

```
10| var app = express();
```

bin/www

problem-based-packs.insecure-transport.js-node.using-http-server.using-http-server

Checks for any usage of http servers instead of https servers. Encourages the usage of https protocol instead of http, which does not have TLS and is therefore unencrypted. Using http can lead to man-in-the-middle attacks in which the attacker is able to read sensitive

 Add Context...

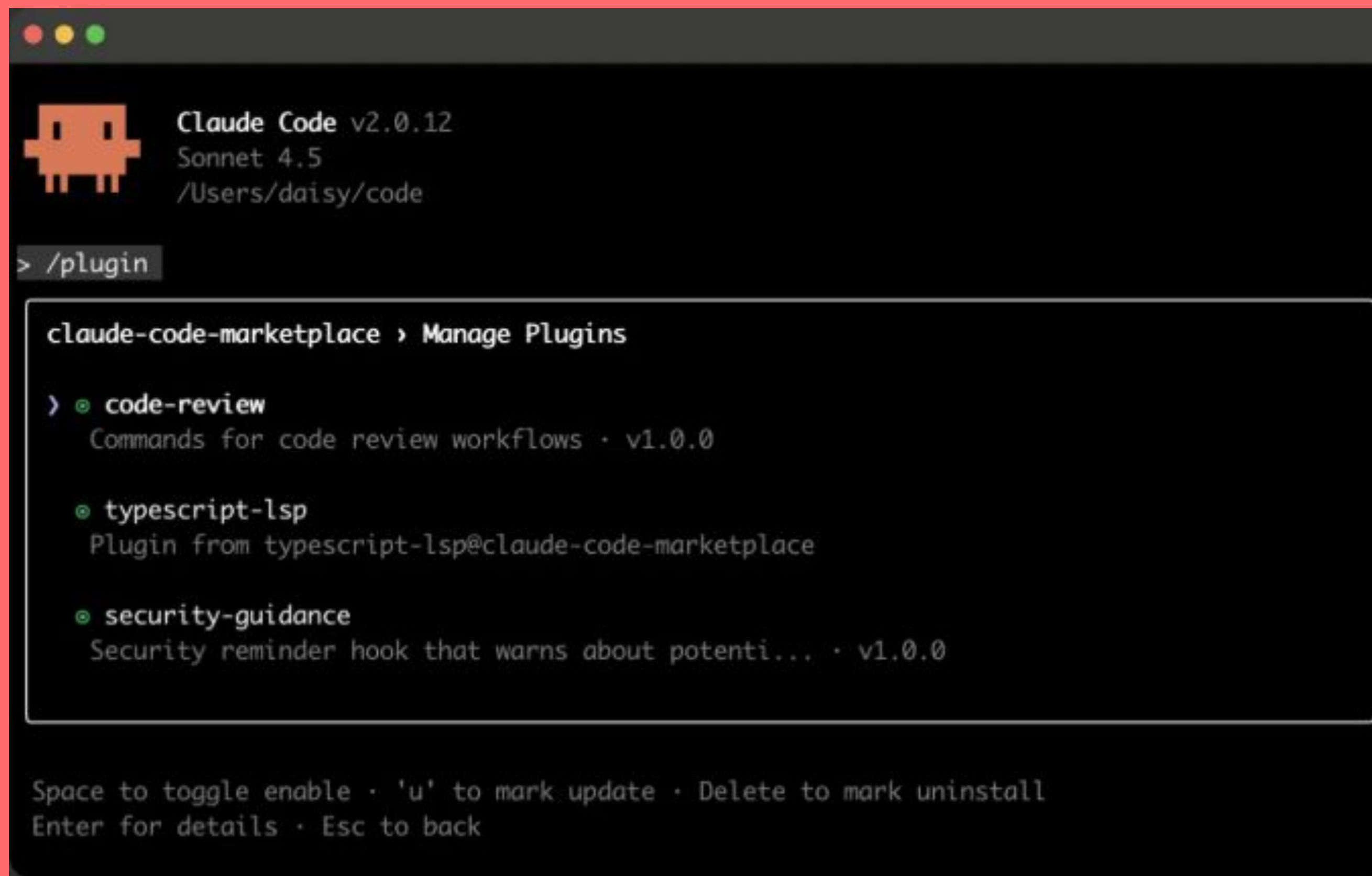
I have a semgrep report here from a scan of my codebase. Help me triage these findings, only focus on the high or critical severity first. Suggest some ways to fix them and propose changes to my code, as well as any ways I can prevent these in the future

Agent ▾ GPT-5 mini ▾



Security plugins & skills:

<https://composio.dev/content/claude-code-plugin>



```
Claude Code v2.0.12
Sonnet 4.5
/Users/daisy/code

> /plugin

claude-code-marketplace › Manage Plugins

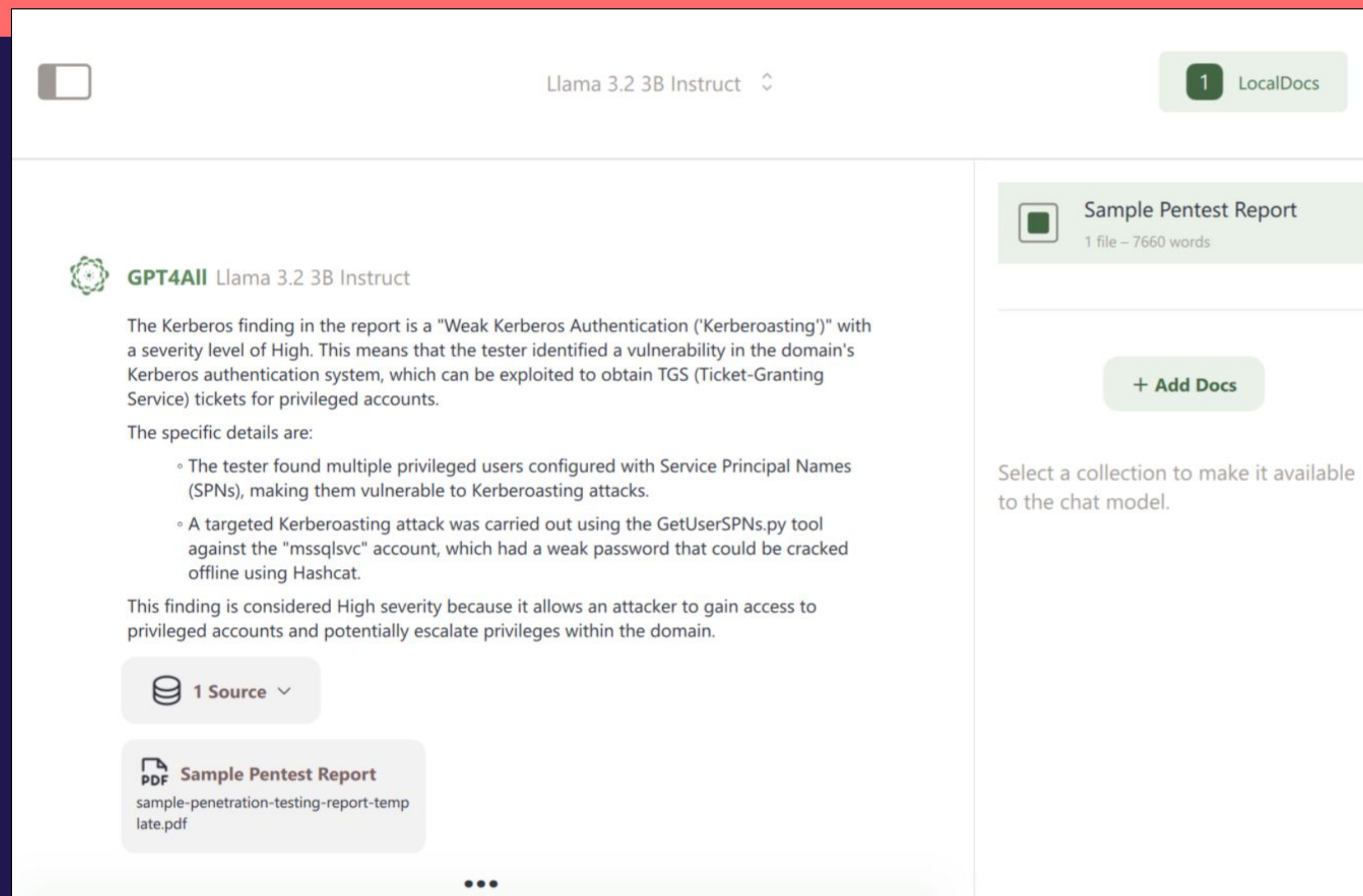
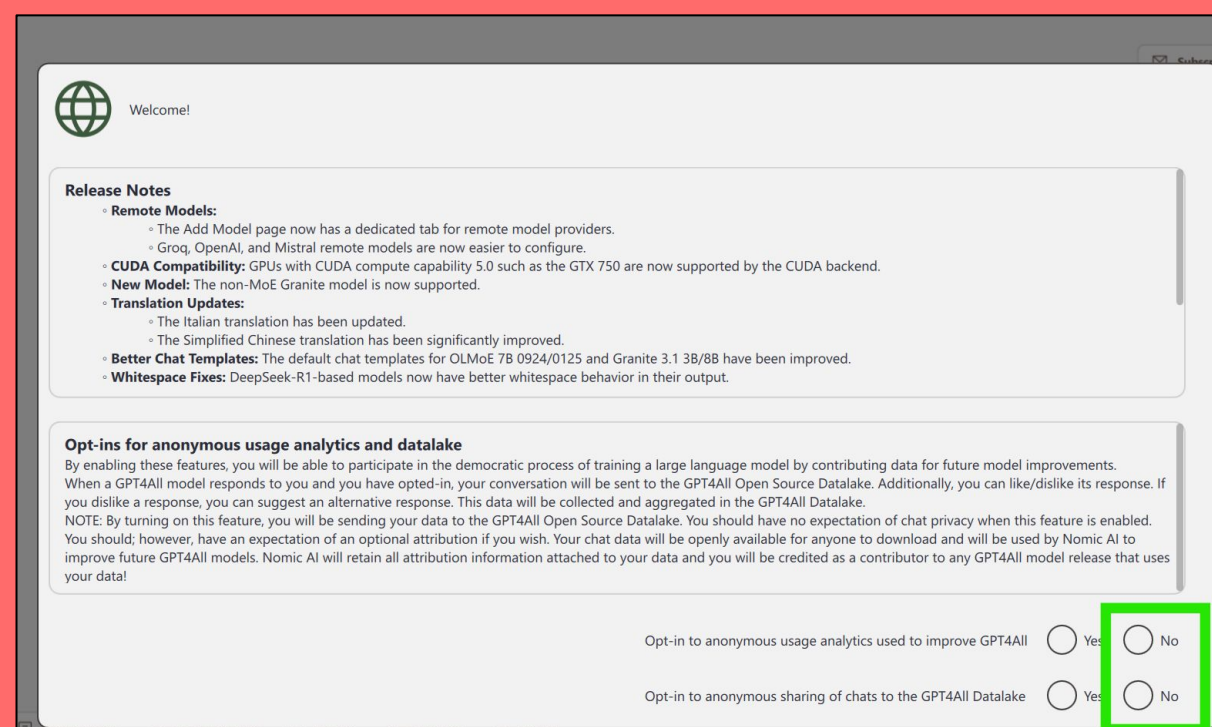
> ● code-review
  Commands for code review workflows · v1.0.0

  ● typescript-lsp
    Plugin from typescript-lsp@claude-code-marketplace

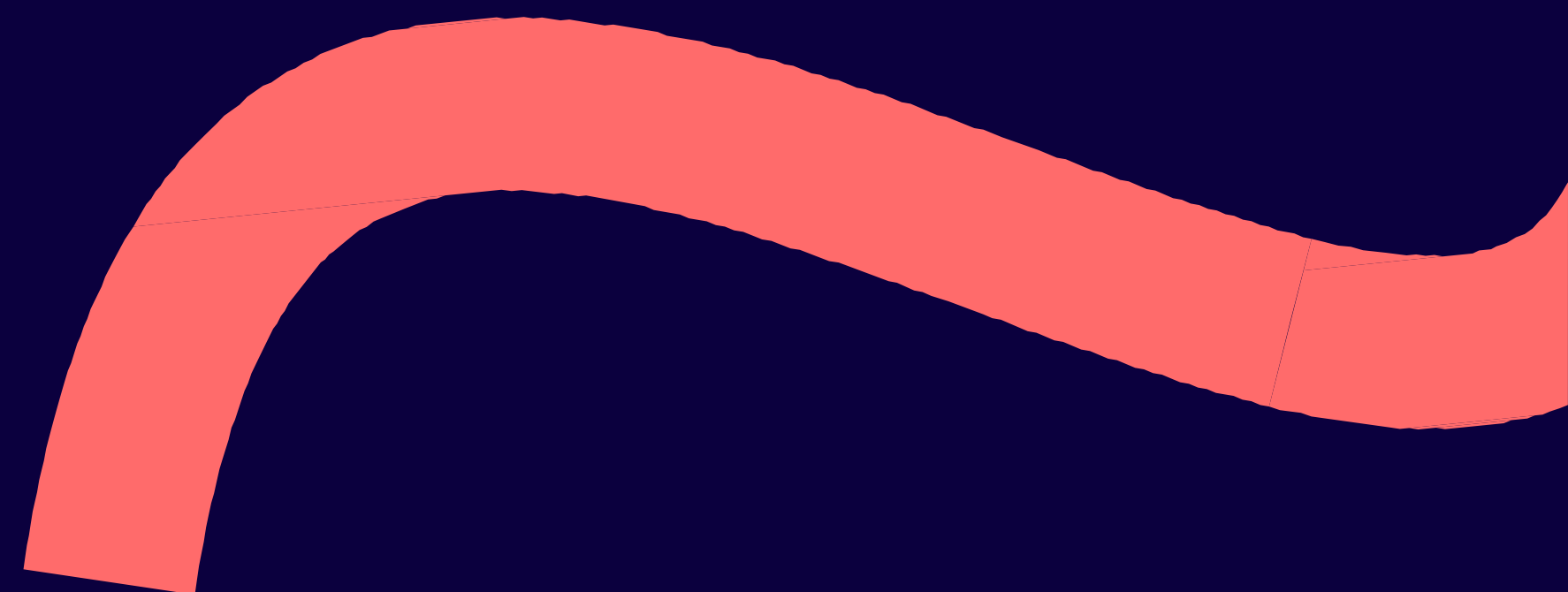
  ● security-guidance
    Security reminder hook that warns about potenti... · v1.0.0

Space to toggle enable · 'u' to mark update · Delete to mark uninstall
Enter for details · Esc to back
```


Offline AI w/ Ollama or GPT4All:



7) AI-assisted coding



How to do 'AI-assisted coding'

Infobip Shift 2026
Zadar, Croatia

- Instead of having the AI write everything, use it as a companion to help you troubleshoot tough errors
- Try to code out the key functions as you normally would manually, but then ask for help when you get tough errors or are exploring new tech stacks
- Get help with troubleshooting and ask for secure architecture advice
- Essentially: treat AI as an advisor instead of outsourcing all your work

Examples:

Help me troubleshoot this error. I am building an application with nodejs and I need to figure out where this issue came up:

```
All tests
passed./project/test/node_modules/mocha/lib/runner.js:
873
throw err;
^Error [ERR_INTERNAL_ASSERTION]: This is caused by
either a bug in Node.js or incorrect usage of Node.js
internals.Please open an issue with this stack trace at
https://github.com/nodejs/node/issues

at assert (internal/assert.js:14:11)
at process.target._send (internal/child_process.js:689:5)
at process.<anonymous>
(internal/child_process.js:646:12)
```



That particular error is a bit of a "blue screen of death" for Node.js processes. While it claims it could be a bug in Node itself, it is much more frequently caused by **attempting to send data to a child process that has already closed or is in an unstable state.**

Since this is happening during a Mocha test run, the culprit is usually how your tests handle asynchronous teardowns or IPC (Inter-Process Communication).

 UNIVERSITY OF NORTH CAROLINA

```
def hash_password(password: str) -> str:
    return hashlib.md5(password.encode("utf-8")).hexdigest()
```

```
@app.post("/register")
def register(user: User):
    if user.username in users:
        raise HTTPException(status_code=400,
detail="Username already exists")
```


Examples:

Give me a bash one liner that does a regex for internal IP addresses in a log file

You can use `grep -E` with a regex for private/internal IPs. Here's a compact **one-liner**:

bash

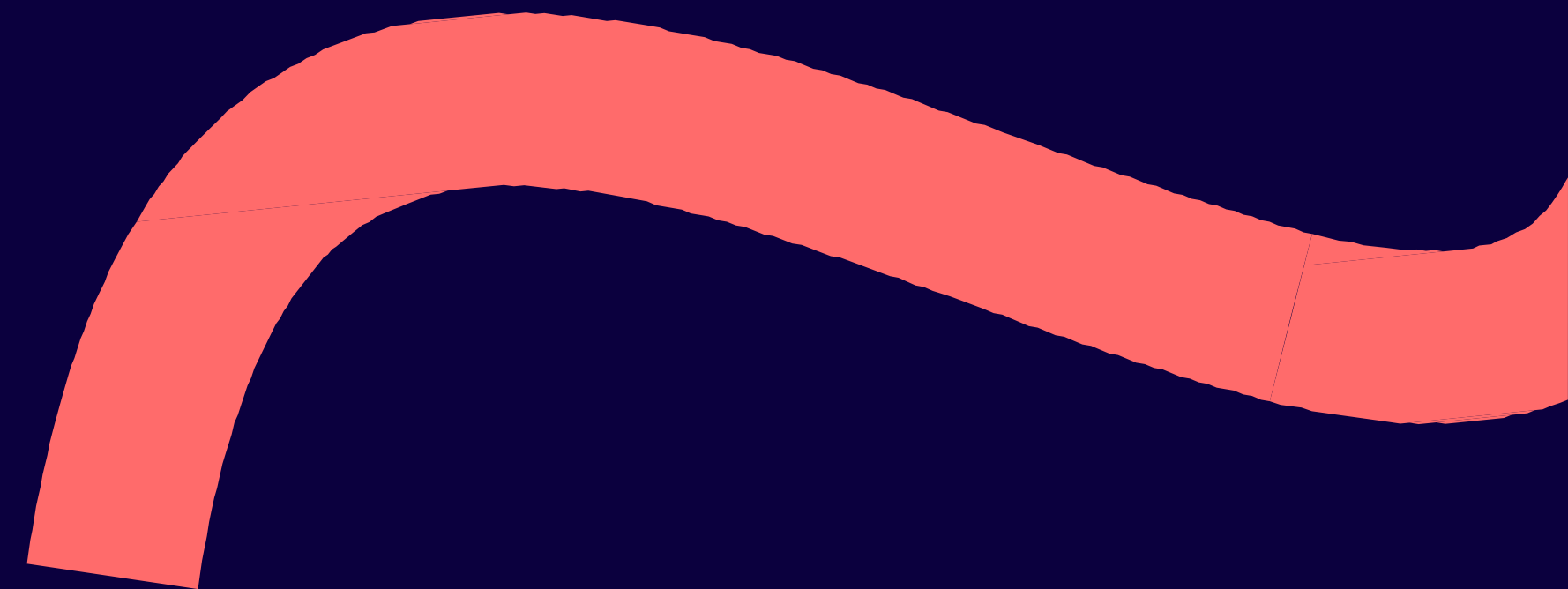
 Copy code

```
grep -Eo '((10|127)\.([0-9]{1,3}\.){2}[0-9]{1,3}|172\.(1[6-9]|2[0-9]|3[0-1])\.([0-9]{1,3}\.){0-9})'
```

✓ Explanation:

- `-E` → extended regex
- `-o` → print only the matching part
- Matches:
 - `10.x.x.x`
 - `127.x.x.x` (loopback)
 - `172.16.x.x` – `172.31.x.x`
 - `192.168.x.x`

8) Further reading + workshop info

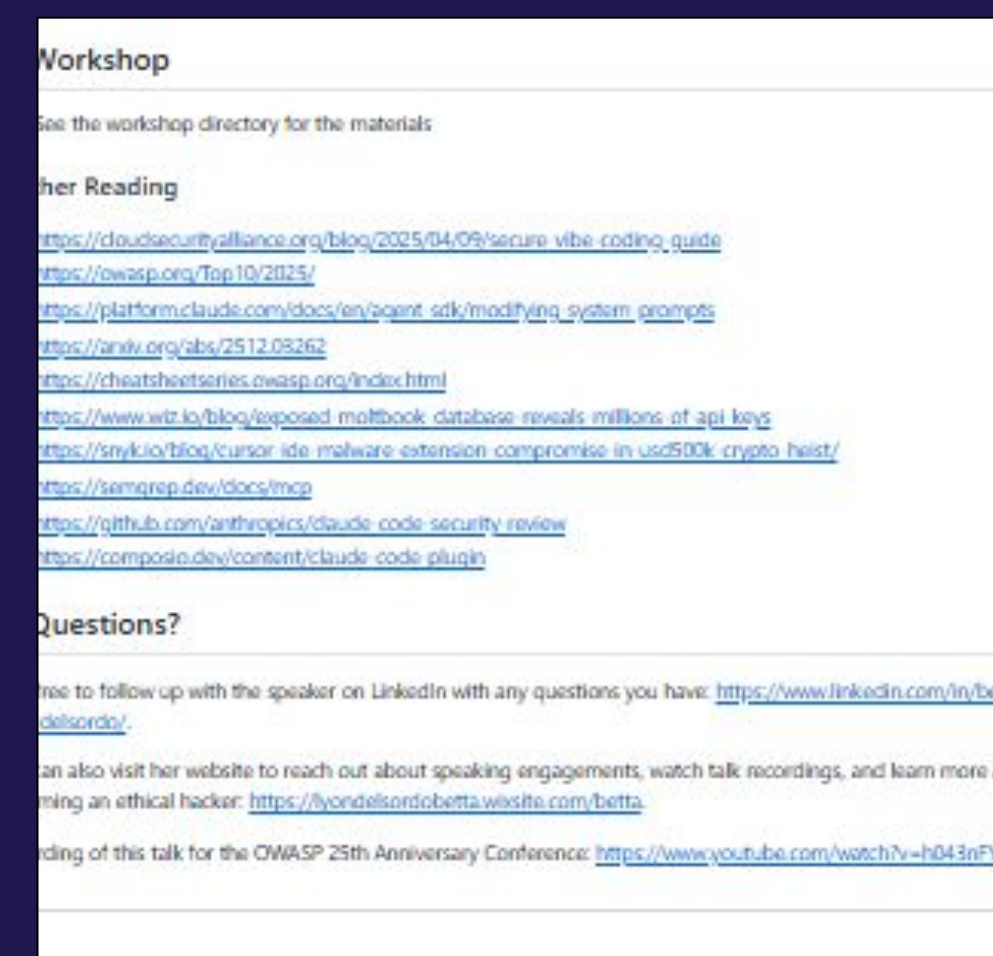


Check out my
workshop repo
and let me know
how it goes...

[https://github.com
/Betta-Lyon-Delso
rdo/insecure-vibe
s/](https://github.com/Betta-Lyon-Delso-rdo/insecure-vibes/)



Scan for the workshop
or go here



Articles & papers for
further reading linked
on the repo...

Let's connect!

Betta Lyon Delsordo

<https://www.linkedin.com/in/betta-lyon-delsordo/>

